What Quantization Really Costs on a Laptop

Speed, memory, accuracy and energy of local LLMs on Apple Silicon

Anthony Vang

ESILV, Data & AI Engineering · November 2026

Abstract. Weight quantization is the standard way to run large language models on consumer hardware, and is commonly described as nearly free in quality and cheaper in energy. We test both claims on a fanless MacBook Air (Apple M5, 16 GB) across 20 configurations: two models of the Qwen2.5 family (1.5B and 3B parameters), seven llama.cpp quantization levels from F16 to Q2_K, and three MLX levels. For each configuration we measure decode and prefill throughput, peak memory, energy per token from hardware counters, GSM8K accuracy on 300 problems, WikiText-2 perplexity, KL divergence from the F16 model, and throughput under 15 minutes of sustained load. Down to about 4 bits per weight, accuracy is statistically unchanged (3B: 85.0% in F16, 84.7% in Q4_K_M), while throughput rises 2.5×. Energy per token, however, falls by only 28%, because power draw rises with speed. Below 4 bits the smaller model degrades much faster, and a 3B model at 3 bits outperforms a 1.5B model at full precision with 35% less memory.

1 Introduction

Running a language model locally keeps data on the device, removes network latency and per-request cost, and is increasingly the default on laptops and phones. The binding constraint is memory: a 3B-parameter model needs about 6 GB in 16-bit precision. Quantization reduces the precision of the weights to 8, 4 or fewer bits, shrinking the model accordingly.

Two claims usually accompany this choice: that 4-bit quantization costs little or no quality, and that it saves energy roughly in proportion to its speed-up. This report measures both, end to end, on a single machine, and identifies the configurations worth using. It makes three contributions:

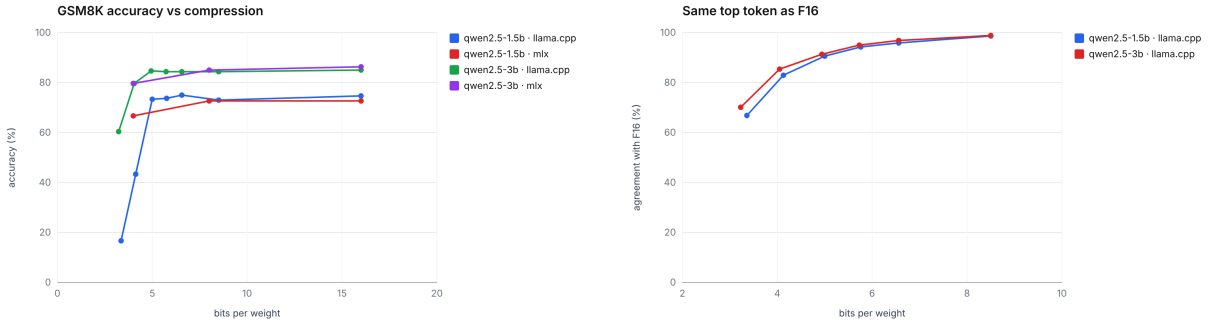
- a systematic sweep of quantization levels and engines with *measured* accuracy, rather than perplexity alone;
- per-token energy read from Apple's hardware energy counters, without elevated privileges;
- sustained-load measurements on a passively cooled laptop, where short benchmarks are known to be optimistic.

2 Experimental setup

Hardware. MacBook Air with an Apple M5 chip, 16 GB of unified memory, passive cooling, on AC power.

Models and engines. Qwen2.5-1.5B-Instruct and Qwen2.5-3B-Instruct, from the official GGUF releases for llama.cpp (F16, Q8_0, Q6_K, Q5_K_M, Q4_K_M, Q3_K_M, Q2_K) and the MLX community conversions (bf16, 8-bit, 4-bit). Compression is reported as effective *bits per weight* (file size $\times 8$ / parameter count), because K-quant labels understate the true size: Q4_K_M is about 4.9–5.0 bits per weight.

Tooling. All measurements are taken by `jpt`, a command-line tool written for this study using only the Python standard library. It manages the inference servers, stores every measurement in SQLite with its hardware and software context, and regenerates all tables and figures from the database. Sweeps are resumable and covered by unit tests run in continuous integration.



(a) GSM8K accuracy. **(b)** Agreement with the F16 model.

Figure 1. Quality against effective bits per weight.

3 Methodology

Throughput. Three prompts cover different regimes: a long-form explanation and a code-generation task (256 output tokens each) and the summary of a long input (64 output tokens). After a discarded warm-up, each prompt is run three times with greedy decoding and a fixed output length; the median is reported with its coefficient of variation (CV). A 10-second pause separates trials, and each configuration starts only once the macOS thermal state is back to nominal. Prompt caching is defeated so that prefill is paid on every trial.

Energy. Apple's IOReport *Energy Model* channels expose cumulative energy counters for the CPU, GPU, neural engine and DRAM. The energy of a request is the difference between two snapshots taken immediately before and after it, which avoids sampling and integration error. Idle power is measured before each configuration, once background activity has settled. Energy per token is total SoC energy divided by generated tokens.

Memory. With Metal, weights live in wired GPU memory that is not attributed to the server process. Peak memory is therefore the increase in system wired memory during the request plus the process footprint.

Quality. *GSM8K*: the first 300 problems of the test split, through each engine's chat template, scored by exact match on the final number, with 95% Wilson intervals (about ±4–5 points). *Perplexity* and *KL divergence*: `llama-perplexity` on 40 WikiText-2 chunks of 512 tokens; the KL divergence compares each quantized model's next-token distribution with that of the F16 model, and we also report how often both rank the same token first.

Selection. A configuration is on the Pareto frontier if no other configuration is at least as good on accuracy, decode throughput and energy per token, and strictly better on one.

4 Results

Table 1 summarizes all 20 configurations.

4.1 Accuracy is preserved down to about 4 bits

On 300 GSM8K problems, the 3B model scores 85.0% in F16 and between 84.3% and 84.7% from Q8_0 to Q4_K_M; the 1.5B model stays between 73.0% and 75.0% over the same range. These differences are well within the confidence intervals. The degradation is nevertheless measurable at the token level: Q4_K_M ranks the same next token first as F16 91% of the time, with a mean KL divergence of 0.03 (Figure 1). Below 4 bits the drift accelerates and accuracy follows: 79.7% then 60.3% for the 3B model at Q3_K_M and Q2_K.

Table 1. Main results. Decode throughput and energy are medians over 9 requests. ⋆ marks the Pareto frontier (accuracy, throughput, energy). Perplexity and KL are measured for llama.cpp only.

Model	Quant.	Bits/w	Tok/s	J/tok	Mem. (GB)	GSM8K (%)	PPL	Same top (%)
1.5B	F16	16.00	37.4	0.237	3.37	74.7	9.17	100.0
	Q8_0	8.50	64.7	0.158	1.73	73.0	9.17	98.7
	Q6_K⋆	6.56	72.9	0.170	1.91	75.0	9.19	95.9
	Q5_K_M	5.76	72.5	0.201	1.48	73.7	9.23	94.3
	Q4_K_M⋆	5.00	83.1	0.137	1.70	73.3	9.57	90.6
	Q3_K_M	4.13	80.7†	0.165	1.57	43.3	10.28	82.9
	Q2_K	3.36	120.2†	0.144	1.11	16.7	15.21	66.8
	MLX bf16	16	40.3	0.194	7.55‡	72.7	–	–
	MLX 8-bit	8	68.8	0.191	5.19‡	72.7	–	–
	MLX 4-bit⋆	4	122.3	0.109	3.79‡	66.7	–	–
3B	F16	16.00	21.2	0.366	6.34	85.0	8.07	100.0
	Q8_0	8.50	36.5	0.287	3.68	84.3	8.07	98.8
	Q6_K	6.56	44.5	0.343	2.99	84.3	8.10	96.9
	Q5_K_M	5.73	49.2	0.367	2.70	84.3	8.16	95.0
	Q4_K_M⋆	4.94	53.3	0.262	2.47	84.7	8.32	91.3
	Q3_K_M	4.05	56.1	0.293	2.15	79.7	8.80	85.4
	Q2_K	3.23	64.2	0.247	1.83	60.3	12.49	70.1
	MLX bf16⋆	16	20.7	0.355	13.26‡	86.3	–	–
	MLX 8-bit⋆	8	37.7	0.245	7.98‡	85.0	–	–
	MLX 4-bit⋆	4	56.4	0.189	5.35‡	79.7	–	–

† CV above 10%: indicative only. ‡ Likely overestimated, as the wired-memory delta includes MLX's buffer cache.

4.2 A larger model compressed harder beats a smaller one

The smaller model collapses much earlier: at Q3_K_M, the 1.5B model falls to 43.3% on GSM8K while the 3B model holds 79.7%. The 3B model at Q3_K_M (2.2 GB) thus outperforms the 1.5B model at full precision (3.4 GB, 74.7%) with 35% less memory, and stays closer to its own F16 reference at every quantization level. Under a fixed memory budget, the largest model that fits after quantization is the better choice.

4.3 Speed scales with compression, energy much less

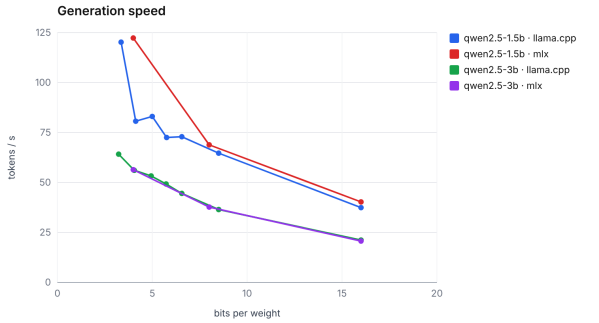
From F16 to Q4_K_M, decode throughput of the 3B model rises from 21.2 to 53.3 tokens/s ($2.5\times$), but energy per token falls only from 0.366 to 0.262 J (-28%). Decoding is memory-bandwidth bound: every weight is read once per generated token, so smaller weights are read faster. The chip, however, works harder: average power rises from 7.8 W to 13.4 W. Since energy per token is power divided by throughput, most of the speed gain is absorbed by the additional power (Figure 2).

4.4 The engine matters as much as the format

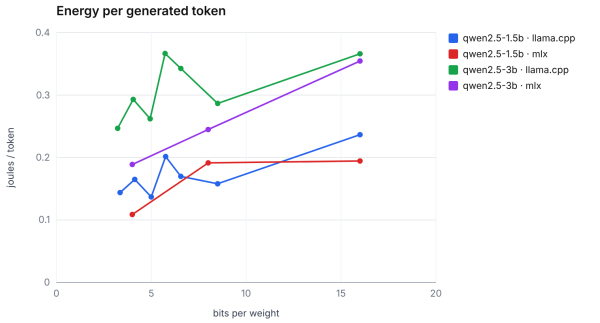
At 4 bits on the 3B model, MLX consumes 28% less energy per token than llama.cpp (0.189 vs 0.262 J) at a similar speed, but its simpler 4-bit scheme costs 5 points on GSM8K (79.7% vs 84.7%), and 7 points on the 1.5B model. llama.cpp's K-quants allocate more precision to the most sensitive tensors, which preserves accuracy at the same nominal bit width. Both engines contribute configurations to the Pareto frontier (Figure 3).

4.5 Short benchmarks overstate sustained speed

Under 15 minutes of continuous generation, the passively cooled chip throttles: Q4_K_M slows from 54 to 45 tokens/s (-16%) and F16 by 10%. The faster configuration draws more power

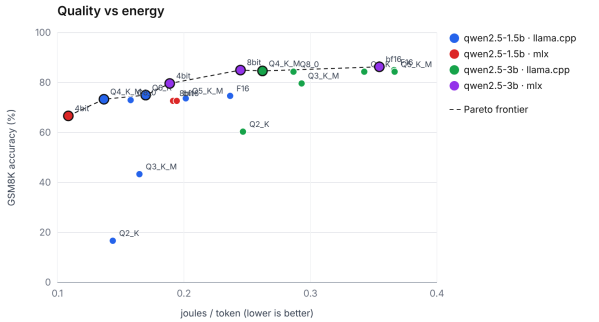


(a) Decode throughput.

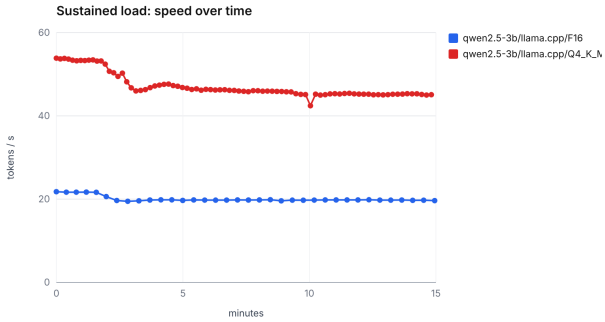


(b) Energy per generated token.

Figure 2. Throughput and energy against effective bits per weight.



(a) Accuracy against energy; dashed: Pareto frontier.



(b) Throughput under sustained load.

Figure 3. Trade-offs and sustained behaviour.

and is throttled more, so benchmarks of a few seconds overstate the speed a user will observe in prolonged use (Figure 3).

5 Recommendation

On this machine, **Qwen2.5-3B in Q4_K_M with llama.cpp** is the configuration to deploy: accuracy equal to F16 within error (84.7% vs 85.0%), 2.5× the throughput, 2.5 GB of memory, and a position on the Pareto frontier. Where energy matters more than a few points of accuracy, MLX 4-bit is the most frugal option.

6 Limitations and future work

- **Scope.** One machine, one model family and two small sizes; results may not transfer to 7B+ models or actively cooled hardware.
- **Energy** covers the whole SoC rather than the model alone; idle subtraction and medians reduce but do not remove background noise.
- **Two configurations** (1.5B Q3_K_M and Q2_K) remain unstable in throughput, and MLX peak memory is likely overestimated.

Future work: a 7–8B model and a second model family, the full GSM8K test set (1,319 problems), and the energy split between prefill and decode at long context lengths.

Reproducibility

Code, raw data, the full results table and the methodology are available in the `joules-per-token` repository. Every table and figure in this report is regenerated from the measurement database by `jpt report`.